PF01 — LABORATORIO 01

Primeros pasos con Python

Aprendiz	Yohan Fernando Cuellar Ospino
Ficha	3388087
Repositorio personal	https://github.com/FIRE31690/Laboratorios_Python
Fecha del informe	23 de octubre de 2026

Propósito: documentar los ejercicios de variables, tipos de datos, operadores, conversiones y entrada de información, junto con la aplicación integradora de cotización de compra.

Alcance y honestidad de la evidencia: se verificó localmente el código preparado para esta entrega y se ejecutaron cuatro pruebas automatizadas, además de dos ejecuciones de consola. No se inventan hashes ni enlaces de commits. El historial remoto de GitHub no se incorporó a esta versión; debe actualizarse después de publicar los archivos y revisar los commits reales.

Repositorio indicado por el aprendiz

Abrir Laboratorios_Python en GitHub

1. Actividades y funcionalidades desarrolladas

Se organizaron los archivos suministrados dentro de **laboratorio-01/**, conservando los ejercicios de práctica y separándolos de la aplicación integradora y sus pruebas.

Ejercicios incluidos: saludo, variables y tipos, perfil del aprendiz, conversor de temperatura, conversión de unidades, operadores y expresiones, cálculo de compra, promedio, salario, conversión de tiempo, factura simple, datos de usuario, suma interactiva y tipo de entrada.

Cotizador: solicita nombre del cliente y producto, precio unitario, cantidad, descuento e impuesto. Calcula subtotal, valor del descuento, base después del descuento, impuesto aplicado a la base y total final. Los porcentajes se restringen al rango de 0 a 100, el precio debe ser positivo, la cantidad un entero mayor que cero y los campos de texto no pueden estar vacíos.

Manejo de entradas inválidas: las conversiones numéricas usan bloques try/except para volver a solicitar el dato cuando se introduce texto no numérico. Se aceptan decimales escritos con punto o coma.

Documentación: se agregaron README.md general y específico del laboratorio, con estructura, requisitos, comandos de ejecución, fórmulas y comando para pruebas.

2. Estado del trabajo

Elemento	Estado
Organización de archivos en laboratorio-01/	Completado en el paquete local preparado.
Cotizador de compra y validaciones	Implementado; ejecución de ejemplo comprobada.
Pruebas automatizadas de cálculos	4 pruebas aprobadas localmente.
Prueba de entrada inválida	Se introdujo texto en el precio; el programa informó el error y volvió a solicitarlo.
README y documentación	Preparados.
Publicación/actualización de GitHub	Pendiente de confirmar que estos archivos se subieron al repositorio indicado.
Historial de commits reales	Pendiente de completar con hashes, fechas, enlaces y mensajes verificados del repositorio.

Dificultad identificada: la versión original del cotizador convertía directamente las entradas con float() e int(); si el usuario escribía letras, el programa terminaba con ValueError. Se incorporó validación y repetición de la pregunta. También se separó el cálculo en una función para probarlo de forma independiente.

Pendiente antes de entregar: publicar los archivos y sustituir la sección de seguimiento de Git por commits que existan realmente. El estado local preparado no demuestra por sí mismo que el repositorio remoto ya esté actualizado.

3. Evidencias de pruebas

Las pruebas siguientes se ejecutaron en el entorno local de preparación usando Python 3. Se verificaron tanto las fórmulas como la reacción a una entrada inválida.

3.1 Pruebas automatizadas

```
Comando:
python3 -m unittest discover -s tests -v

Resultado observado:
test_descuento_e_impuesto ... ok
test_descuento_total ... ok
test_impuesto_cero ... ok
test_sin_descuento_ni_impuesto ... ok

Ran 4 tests in 0.000s
OK
```

3.2 Caso numérico de consola

Dato	Valor
Cliente / producto	Yohan / Cuaderno
Precio unitario / cantidad	$1.000 / 2
Descuento	10 %
Impuesto	19 %
Subtotal	$2.000,00
Descuento calculado	$200,00
Base después del descuento	$1.800,00
Impuesto sobre la base	$342,00
Total esperado y observado	$2.142,00

3.3 Validación de entrada

Se ejecutó el programa introduciendo primero “abc” en el campo de precio y después 1000. El programa mostró un mensaje de entrada inválida y volvió a preguntar el precio; continuó el cálculo y presentó el mismo total de $2.142,00. Esta evidencia corresponde a una ejecución de consola, no a una captura de pantalla.

Fórmulas aplicadas

```
subtotal = precio_unitario * cantidad
valor_descuento = subtotal * descuento_pct / 100
base = subtotal - valor_descuento
valor_impuesto = base * impuesto_pct / 100
total = base + valor_impuesto
```

4. Respuestas de análisis situacional

Pregunta 1. ¿Qué harías si el programa muestra un error porque intentas utilizar una variable que no ha sido declarada o cuyo nombre está escrito incorrectamente?

Qué haría: leería el traceback y ubicaría el archivo y la línea que señalan el error, normalmente un NameError. Revisaría si la variable se asignó antes de utilizarla y si el nombre coincide exactamente, incluidas mayúsculas y minúsculas. **Por qué:** Python distingue los identificadores y no puede resolver una variable que no existe en ese ámbito. **Cómo:** corregiría el nombre o declararía la variable antes de la instrucción que la usa, sin crear variables innecesarias para ocultar el error. **Comprobación:** volvería a ejecutar el programa con el mismo caso que fallaba y comprobaría que imprime el valor esperado; después ejecutaría los demás ejercicios afectados.

Pregunta 2. ¿Qué harías si un usuario introduce letras cuando el cotizador solicita un precio mediante input()?

Qué haría: controlaría la conversión de la entrada con try/except ValueError y volvería a solicitar el precio si no se puede convertir a número. **Por qué:** input() siempre devuelve texto y float("abc") no puede convertir esa cadena, por lo que Python lanza ValueError. **Cómo:** pondría únicamente la conversión dentro del try, mostraría un mensaje claro en except y validaría además que el precio sea positivo. **Comprobación:** introduciría primero letras y luego un precio válido; el programa debe seguir ejecutándose, volver a preguntar y calcular el total correcto sin cerrarse.

Pregunta 3. ¿Qué harías si el cotizador calcula correctamente el subtotal, pero el descuento aplicado no corresponde al porcentaje indicado por el usuario?

Qué haría: revisaría el valor leído del porcentaje, la conversión a proporción y la fórmula del descuento. **Por qué:** el porcentaje debe dividirse entre 100; por ejemplo, 10 % equivale a 0,10, no a 10. También debe descontarse el valor del descuento al subtotal antes de calcular el impuesto si esa es la regla definida. **Cómo:** comprobaría que se usa descuento = subtotal * porcentaje / 100, base = subtotal - descuento e impuesto = base * impuesto / 100. **Comprobación:** probaría precio 1000, cantidad 2, descuento 10 % e impuesto 19 %: subtotal 2000, descuento 200, base 1800, impuesto 342 y total 2142.

Pregunta 4. ¿Qué harías si necesitas calcular el promedio de varios valores y el resultado obtenido no coincide con el cálculo manual?

Qué haría: comprobaría la lista de valores, el tipo de dato de cada entrada, la suma y la cantidad de valores usados como divisor. **Por qué:** un promedio aritmético es la suma de los valores dividida por el número de valores; usar división entera, omitir un dato o dividir por un número incorrecto cambia el resultado. **Cómo:** convertiría las notas a int o float según corresponda, usaría paréntesis en (nota_1 + nota_2 + nota_3) / 3 y evitaría // para el promedio. **Comprobación:** calcularía un ejemplo pequeño manualmente y lo compararía con la salida del programa; por ejemplo (4,2 + 3,8 + 4,5) / 3 = 4,1666… aproximadamente.

Pregunta 5. ¿Qué harías si después de modificar el cotizador deja de funcionar una operación que anteriormente producía el resultado correcto?

Qué haría: identificaría el cambio comparando el historial de commits, revisaría el diff y reproduciría el fallo. **Por qué:** cada commit registra un estado del proyecto y permite acotar qué modificación pudo introducir la regresión. **Cómo:** en GitHub abriría el historial de commits, entraría al commit sospechoso y compararía los archivos cambiados; localmente también podría usar git log --oneline y git diff COMMIT_ANTERIOR COMMIT_ACTUAL. Corregiría el error en un nuevo commit o revertiría el cambio problemático si fuera necesario, evitando borrar modificaciones no relacionadas. **Comprobación:** ejecutaría el caso que fallaba y todas las pruebas automatizadas. Documentaría el hash real, el motivo del cambio y el resultado; no se consignan hashes en este informe porque aún no se han verificado.

5. Seguimiento de commits y entrega

El requisito del laboratorio pide commits progresivos que evidencien la creación de archivos, la implementación de funcionalidades y las correcciones. No se incorporan hashes ficticios: la tabla debe completarse después de publicar los cambios y consultar el historial real del repositorio.

Hash y enlace real	Cambio	Motivo	Comprobación
Pendiente de verificar	Organización inicial de ejercicios y README	Estructurar el laboratorio	Revisar archivos en GitHub
Pendiente de verificar	Implementación del cotizador	Calcular subtotal, descuento, impuesto y total	Ejecutar el cotizador
Pendiente de verificar	Validaciones y pruebas	Evitar cierre por entradas inválidas y verificar fórmulas	4 pruebas unitarias y prueba de entrada

6. Pasos pendientes para la entrega final

1. Copiar el contenido del paquete al repositorio local asociado a FIRE31690/Laboratorios_Python, conservando los demás laboratorios.

2. Revisar con git status que solo se incluyan los archivos deseados.

3. Crear commits reales y descriptivos por etapas; por ejemplo, separar organización inicial, cotizador y validaciones/pruebas, siempre que cada commit corresponda a cambios efectivamente realizados.

4. Ejecutar las pruebas otra vez desde la copia del repositorio y hacer push a GitHub.

5. Abrir el repositorio en el navegador y comprobar que los archivos, el README y los commits están disponibles; actualizar la tabla de este informe con hashes y enlaces reales.

6. Entregar el PDF y el enlace individual de GitHub.

Nota final: el paquete local preparado incluye código, documentación y pruebas. Este informe describe las verificaciones realizadas en ese paquete; la publicación remota y los commits deben verificarse por separado antes de presentar la versión definitiva.